ENGINEERING ONBOARDING & ARCHITECTURE MANUAL

BusBooking Web API (.NET 10 & SQL Server)

Complete Specification Prompt, Conversation History, Clean Architecture Guide & Local Machine Setup Instructions

---

Solution Path:
C:\Projects\BusBookingApi\BusBooking.slnx

Target Framework: .NET 10.0 / C# 14

Database Engine: Microsoft SQL Server (BusBookingDb)

Security Model: JWT Bearer + RBAC (Admin, Manager, Associate)

Table of Contents

1. **Section 1: The Complete Original Prompts & Conversation Evolution**
   - 1.1 Initial User Request Prompt
   - 1.2 The Complete Architectural Master Prompt (Verbatim)
   - 1.3 Chronological Evolution & Engineering Refinements During Session
2. **Section 2: Detailed Explanation of What the Application Does**
   - 2.1 Business Purpose & Core Functional Flows
   - 2.2 Clean Architecture Layer Breakdown (4 Projects)
   - 2.3 Database Schema, Covering Indexes & Concurrency-Safe Indexed View
   - 2.4 Enterprise Non-Functional Capabilities (NFRs)
   - 2.5 Complete API Endpoints & Role-Based Authorization (RBAC) Matrix
3. **Section 3: Step-by-Step Instructions to Run the Application Locally**
   - 3.1 Machine Prerequisites
   - 3.2 Step 1 — Navigate to the Workspace
   - 3.3 Step 2 — Configure Connection String & JWT Settings
   - 3.4 Step 3 — Initialize & Seed the SQL Server Database
   - 3.5 Step 4 — Build & Run the Application
   - 3.6 Step 5 — Using Swagger UI & Default Seeded Role Credentials
   - 3.7 Step 6 — Running the Automated PowerShell Verification Suite
   - 3.8 Step 7 — Troubleshooting Guide for New Engineers

Section 1: The Complete Original Prompts & Conversation Evolution

1.1 Initial User Request Prompt

> "create a prompt which will help me build a web api in dotnet 10 and sql server database with all the best practices and highly performant. this is a bus booking api and the details can be found at the site https://freeprojectapi.com/apis/bus-booking, the end points should look like this and accordingly the db tables should be created, this application should have health checks + matrics, observability, rate limiting, api versioning, proper logging, caching, feature management, exception handling management, resilience"

1.2 The Complete Architectural Master Prompt (Verbatim)

Below is the complete Master Prompt generated from the initial request and executed across Steps 1–4 to build the solution:

> **Role & Objective:** You are a Principal .NET Solutions Architect and High-Performance Systems Engineer. Your task is to design and implement a production-ready, ultra-low latency, and resilient Bus Ticket Booking Web API built with **.NET 10** and **Microsoft SQL Server**.
>
> The business domain and contract specifications mirror the public API contract from FreeProjectAPI (`https://freeprojectapi.com/apis/bus-booking`). You must build the system using **Clean Architecture**, modern C# 14 idioms, zero-allocation best practices, and enterprise non-functional requirements (Observability, Resilience, Caching, Rate Limiting, Versioning, Feature Management, and Health Checks).

1. Technology Stack & Key Libraries

- **Runtime & Language:** .NET 10 / C# 14
- **Database:** Microsoft SQL Server 2022+
- **Data Access:**
  - **Entity Framework Core 10** for Domain writes, migrations, relationships, and transactional consistency.
  - **Dapper / Compiled No-Tracking Queries** for high-throughput read paths (e.g., bus search and route exploration).
- **Caching:** .NET `HybridCache` (L1 In-Memory + L2 Distributed Redis) with tag-based cache eviction.
- **Resilience:** Polly v8 via `Microsoft.Extensions.Resilience` (Retry, Circuit Breaker, Timeout, Rate Limiter).
- **Observability:**
  - OpenTelemetry (`OpenTelemetry.Extensions.Hosting` , ASP.NET Core, HttpClient, EF Core instrumentation, OTLP Exporter).
  - Custom `System.Diagnostics.Metrics.Meter` for booking KPIs.
  - Serilog with structured logging and OpenTelemetry trace/span correlation.

- **Health Checks:** `AspNetCore.HealthChecks.SqlServer`, custom readiness/liveness probes (`/health/live`, `/health/ready`).
- **Rate Limiting:** `Microsoft.AspNetCore.RateLimiting` (Fixed Window, Sliding Window, Token Bucket).
- **API Versioning:** `Asp.Versioning.Http` and `Asp.Versioning.Mvc.ApiExplorer` (supporting `/api/v{version:apiVersion}/...` with backward-compatible fallback).
- **Feature Management:** `Microsoft.FeatureManagement.AspNetCore`.
- **Exception Handling:** Global `IExceptionHandler` producing standard RFC 7807 `ProblemDetails`.
- **Authentication & Security:** ASP.NET Core JWT Bearer authentication, Refresh Tokens, BCrypt password hashing, and Role-Based Authorization (`Admin`, `Manager`, `Associate`).

2. Relational Database Schema & Data Modeling

1. **`Users`**: `UserId` (INT, IDENTITY, PK), `UserName` (NVARCHAR(100), UNIQUE, NOT NULL), `EmailId` (NVARCHAR(150), UNIQUE, NOT NULL), `FullName` (NVARCHAR(150), NOT NULL), `Role` (NVARCHAR(50), NOT NULL), `PasswordHash` (NVARCHAR(255), NOT NULL), `PasswordSalt` (NVARCHAR(255), NOT NULL), `CreatedDate` (DATETIMEOFFSET, NOT NULL, DEFAULT SYSUTCDATETIME()), `RefreshToken` (NVARCHAR(255), NULL), `RefreshTokenExpiryTime` (DATETIMEOFFSET, NULL), `RowVersion` (ROWVERSION / Concurrency Token).
2. **`BusVendors`**: `VendorId` (INT, IDENTITY, PK), `VendorName` (NVARCHAR(150), NOT NULL), `ContactNo` (VARCHAR(20), NOT NULL), `EmailId` (NVARCHAR(150), NOT NULL), `IsActive` (BIT, NOT NULL, DEFAULT 1).
3. **`BusLocations`**: `LocationId` (INT, IDENTITY, PK), `LocationName` (NVARCHAR(100), NOT NULL), `Code` (VARCHAR(10), NOT NULL, UNIQUE).
4. **`BusAddressLocations`**: `LocationPointId` (INT, IDENTITY, PK), `LocationId` (INT, FK -> BusLocations.LocationId, ON DELETE CASCADE), `LocationTitle` (NVARCHAR(150), NOT NULL), `LocationAddress` (NVARCHAR(500), NOT NULL).
5. **`BusSchedules`**: `ScheduleId` (INT, IDENTITY, PK), `VendorId` (INT, FK -> BusVendors.VendorId), `BusName` (NVARCHAR(100), NOT NULL), `BusVehicleNo` (VARCHAR(50), NOT NULL), `FromLocation` (INT, FK -> BusLocations.LocationId), `ToLocation` (INT, FK -> BusLocations.LocationId), `DepartureTime` (TIME, NOT NULL), `ArrivalTime` (TIME, NOT NULL), `ScheduleDate` (DATE, NOT NULL), `Price` (DECIMAL(18,2), NOT NULL), `TotalSeats` (INT, NOT NULL), `RowVersion` (ROWVERSION).
6. **`BusBookings`**: `BookingId` (INT, IDENTITY, PK), `CustId` (INT, FK -> Users.UserId), `ScheduleId` (INT, FK -> BusSchedules.ScheduleId), `BookingDate` (DATETIMEOFFSET, NOT NULL, DEFAULT SYSUTCDATETIME()), `TotalAmount` (DECIMAL(18,2), NOT NULL), `BookingStatus` (NVARCHAR(30), DEFAULT 'Confirmed').
7. **`BusBookingPassengers`**: `PassengerId` (INT, IDENTITY, PK), `BookingId` (INT, FK -> BusBookings.BookingId, ON DELETE CASCADE), `PassengerName` (NVARCHAR(150), NOT NULL), `Age` (INT, NOT NULL), `Gender` (VARCHAR(10), NOT NULL), `SeatNo` (INT, NOT NULL).

3. Critical Constraints & Performance Indexes

- **Seat Conflict Prevention:** Schema-bound indexed view `dbo.vw_ActiveScheduleSeats` with `UNIQUE CLUSTERED INDEX IX_vw_ActiveScheduleSeats_Schedule_Seat ON dbo.vw_ActiveScheduleSeats(ScheduleId, SeatNo)` plus atomic `SERIALIZABLE` transaction checks so two concurrent users cannot book the same seat on the same schedule.

- **Search Covering Index:** `CREATE NONCLUSTERED INDEX IX_BusSchedules_Search ON BusSchedules(FromLocation, ToLocation, ScheduleDate) INCLUDE (ScheduleId, VendorId, BusName, BusVehicleNo, DepartureTime, ArrivalTime, Price, TotalSeats);`
- **Booked Seats Lookup Index:** `CREATE NONCLUSTERED INDEX IX_BusBookings_Schedule_Status ON BusBookings(ScheduleId, BookingStatus) INCLUDE (BookingId, CustId, BookingDate);`

4. Step-by-Step Execution Plan Used to Build the System

1. **Step 1 — Domain & Database:** Generated the EF Core 10 entities, `BusBookingDbContext`, Fluent configurations, initial migration, and SQL Server DDL script with covering indexes and the concurrency-safe indexed view.
2. **Step 2 — Core Booking & Search Logic:** Implemented CQRS handlers via MediatR, high-throughput Dapper queries (`SearchBusQuery`, `GetAvailableRoutesQuery`, `GetBookedSeatsQuery`), `HybridCache` caching, and atomic `CreateBusBookingCommand`.
3. **Step 3 — API Endpoints & Contracts:** Created the controllers, DTOs, and request validators, and moved the workspace to `C:\Projects\BusBookingApi`.
4. **Step 4 — Enterprise NFR Integration:** Wired `Program.cs` with Serilog, OpenTelemetry metrics/tracing, Polly v8 resilience pipelines, rate limiters, API versioning, feature flags, health checks, and `GlobalExceptionHandler`.

1.3 Chronological Evolution & Engineering Refinements During Session

Stage	User Request / Requirement	Engineering Solution Implemented
1. Workspace Relocation	Move `BusBookingApi` to `C:\Projects` and update all references.	Moved solution to `C:\Projects\BusBookingApi`, verified `BusBooking.slnx` project paths and SQL scripts.
2. Swagger UI	Enable Swagger UI to view and test endpoints in the browser.	Configured `Swashbuckle.AspNetCore` v10 in `Program.cs` with root redirect (`/` -> `/swagger`).
3. Authentication & RBAC	Implement JWT authentication and role-based authorization for roles: `Admin`, `Manager`, `Associate`.	Implemented `JwtTokenService`, configured `AddJwtBearer`, added RBAC policies, and seeded default accounts (`admin`, `manager`, `associate`).
4. Dedicated AuthController	Move all authentication and user authorization endpoints out of `BusBookingController` into a dedicated controller.	Created `AuthController` (`/api/Auth/...`) for `GetAllUsers`, `login`, `AddNewUser`, `UpdateUser`, and `DeleteUserByUserId`; removed them completely from `BusBookingController`.
5. Git Hygiene & Script Reliability	Add a comprehensive `.gitignore` and fix PowerShell script timeouts.	Added root `.gitignore`, untracked `bin/` and `obj/`, added `-UseBasicParsing` and redirected stdio in PowerShell test scripts.

Stage	User Request / Requirement	Engineering Solution Implemented
6. Swagger Bearer Fix	Fix `401 Unauthorized` when authorizing with Bearer token in Swagger UI.	Passed `document` into `new OpenApiSecuritySchemeReference("Bearer", document)` so `swagger.json` emits the security requirement, and added `JwtBearerEvents.OnMessageReceived` to accept both `<token>` and `Bearer <token>`.

Section 2: Detailed Explanation of What the Application Does

2.1 Business Purpose & Core Functional Flows

BusBookingApi is a high-throughput backend Web API that powers an intercity bus ticket reservation system. It supports five core functional workflows:

1. **Authentication & User Lifecycle (`AuthController`):** Users register accounts with passwords hashed via BCrypt (cost factor 12) and log in to receive a signed HMAC-SHA256 JWT Bearer token (120-minute expiry) plus a 7-day refresh token. Administrators and Managers can audit user accounts, and Administrators can remove accounts.
2. **Bus Vendor Management (`BusBookingController`):** Manages bus operator companies (e.g., *Greyhound Express*, *MegaBus Lines*, *PeterPan Coach*), their contact numbers, email addresses, and active status.
3. **Locations & Boarding/Drop-off Points (`BusBookingController`):** Manages city terminals (`BusLocations` with unique station codes such as `NYC`, `BOS`, `DC`, `PHI`) and specific boarding/drop-off points (`BusAddressLocations`) within each city. Location queries are cached in `HybridCache` for 60 minutes.
4. **Schedule Publishing & Fast Bus Search (`BusBookingController`):** Operators publish schedules with origin, destination, travel date, departure/arrival times, price, and total seat capacity. Customers search routes via Dapper queries backed by SQL covering indexes and 2-minute `HybridCache` entries that dynamically compute `availableSeats = totalSeats - bookedSeats`.
5. **Atomic Seat Reservation & Cancellation (`BusBookingController`):** Customers book specific seat numbers for one or more passengers. The system prevents double-booking using both a `SERIALIZABLE` transaction in EF Core and a unique clustered index on the schema-bound view `dbo.vw_ActiveScheduleSeats`.

2.2 Clean Architecture Layer Breakdown (4 Projects)

Project	Path	Responsibilities & Key Components
BusBooking.Domain	`src/BusBooking.Domain`	Core enterprise entities (`User`, `BusVendor`, `BusLocation`, `BusAddressLocation`, `BusSchedule`, `BusBooking`, `BusBookingPassenger`) and domain constants (`UserRole`, `BookingStatus`). Zero external package dependencies.
BusBooking.Application	`src/BusBooking.Application`	CQRS commands/queries via `MediatR`, `FluentValidation` pipeline behavior (`ValidationBehavior`), DTOs, custom exceptions (`SeatAlreadyBookedException`, `NotFoundException`), and OpenTelemetry metrics (`BusBookingMetrics`).

Project	Path	Responsibilities & Key Components
BusBooking.Infrastructure	`src/BusBooking.Infrastructure`	Pooled EF Core 10 `BusBookingDbContext`, entity configurations, migrations, Dapper `SqlConnectionFactory`, `JwtTokenService`, and .NET 10 `HybridCache` registration.
BusBooking.Api	`src/BusBooking.Api`	`AuthController`, `BusBookingController`, RFC 7807 `GlobalExceptionHandler`, JWT Bearer authentication, RBAC policies, Rate Limiting, Polly v8 resilience, OpenTelemetry, Serilog, Health Checks, and Swagger UI.

2.3 Database Schema, Covering Indexes & Concurrency-Safe Indexed View

The SQL Server database `BusBookingDb` contains 7 relational tables and 1 schema-bound indexed view:

- **`dbo.Users`** — Stores user credentials, BCrypt hashes/salts, roles (`Admin`, `Manager`, `Associate`, `Customer`), refresh tokens, and `RowVersion` concurrency token.
- **`dbo.BusVendors`** — Stores bus operator companies and `IsActive` status.
- **`dbo.BusLocations`** — Stores city terminals and unique station codes (`UQ_BusLocations_Code`).
- **`dbo.BusAddressLocations`** — Stores boarding/dropping landmarks linked to `BusLocations` (`ON DELETE CASCADE`).
- **`dbo.BusSchedules`** — Stores scheduled bus trips, pricing, seat counts, and covering index `IX_BusSchedules_Search`.
- **`dbo.BusBookings`** — Stores booking headers linked to customer and schedule with status (`Confirmed` / `Cancelled`).
- **`dbo.BusBookingPassengers`** — Stores individual passenger records and assigned seat numbers (`SeatNo`).
- **`dbo.vw_ActiveScheduleSeats` (Indexed View)** — Schema-bound join of `BusBookings` (where `BookingStatus = 'Confirmed'`) and `BusBookingPassengers` with a **Unique Clustered Index** on `(ScheduleId, SeatNo)`. This guarantees at the database engine level that no two active bookings can ever occupy the same seat on the same schedule, while still allowing a seat to be re-booked if an earlier booking is cancelled.

2.4 Enterprise Non-Functional Capabilities (NFRs)

- **Rate Limiting (`Microsoft.AspNetCore.RateLimiting`):**
  - `fixed-general`: 100 requests per minute (Fixed Window).
  - `sliding-search`: 60 requests per minute across 6 segments (Sliding Window) on `searchBus` / `searchBus2`.
  - `token-booking`: 10-token bucket replenishing 1 token every 2 seconds on `PostBusBooking`.

- **Caching (`HybridCache`):** L1 in-memory cache with stampede protection for locations (60-min TTL) and bus search results (2-min TTL).
- **Resilience (Polly v8 + EF Core Retry):** Exponential backoff with jitter on transient failures and SQL Server retry-on-failure strategy.
- **Observability (OpenTelemetry & Serilog):** Custom meter `BusBooking.Monitoring` tracking `bus_bookings_created_total`, `bus_seats_booked_total`, and `bus_booking_failures_total`, paired with structured Serilog console logging.
- **Global Exception Handling (RFC 7807):** Maps `ValidationException` -> `400 Bad Request`, `NotFoundException` -> `404 Not Found`, `SeatAlreadyBookedException` -> `409 Conflict`, and unexpected exceptions -> `500 Internal Server Error` with `traceId`.

2.5 Complete API Endpoints & Role-Based Authorization (RBAC) Matrix

A. `AuthController` (Base Route: `/api/Auth` and `/api/v1/Auth`)

Method	Endpoint	Access / Roles	Purpose
POST	`/api/Auth/login`	`AllowAnonymous`	Authenticates username & password; returns signed JWT token & user profile.
POST	`/api/Auth/AddNewUser`	`AllowAnonymous`	Registers a new user account with BCrypt password hashing and role normalization.
GET	`/api/Auth/GetAllUsers`	`Admin, Manager`	Returns all registered users (with passwords masked).
POST	`/api/Auth/UpdateUser`	`Authenticated`	Updates user full name, email, role, and optionally re-hashes password.
DELETE	`/api/Auth/DeleteUserByUserId?userId={id}`	`Admin`	Permanently deletes a user account by ID.

B. `BusBookingController` (Base Route: `/api/BusBooking` and `/api/v1/BusBooking`)

Domain Area	Method	Endpoint	Access / Roles
Vendors	GET	`/api/BusBooking/GetBusVendors`	`Admin, Manager, Associate`
Vendors	GET	`/api/BusBooking/GetBusVendorsById?id={id}`	`Admin, Manager, Associate`
Vendors	POST	`/api/BusBooking/PostBusVendor`	`Admin, Manager`
Vendors	PUT	`/api/BusBooking/PutBusVendors?id={id}`	`Admin, Manager`
Vendors	DELETE	`/api/BusBooking/DeleteBusVendor?id={id}`	`Admin`

Domain Area	Method	Endpoint	Access / Roles
Locations	GET	`/api/BusBooking/GetBusLocations`	`AllowAnonymous`
Locations	GET	`/api/BusBooking/GetBusLocationById?id={id}`	`AllowAnonymous`
Locations	POST	`/api/BusBooking/PostBusLocation`	`Admin, Manager`
Locations	PUT	`/api/BusBooking/PutBusLocation?id={id}`	`Admin, Manager`
Locations	DELETE	`/api/BusBooking/DeleteBusLocation?id={id}`	`Admin`
Locations	GET	`/api/BusBooking/getAddressByLocationId?locationid={id}`	`AllowAnonymous`
Locations	POST	`/api/BusBooking/PostBusLocationAddress`	`Admin, Manager`
Schedules	GET	`/api/BusBooking/GetAvailableRoutes`	`AllowAnonymous`
Schedules	GET	`/api/BusBooking/GetBusSchedules?vendorId={id}`	`Admin, Manager, Associate`
Schedules	GET	`/api/BusBooking/GetBusScheduleById?id={id}`	`Admin, Manager, Associate`
Schedules	POST	`/api/BusBooking/PostBusSchedule`	`Admin, Manager`
Schedules	PUT	`/api/BusBooking/PutBusSchedule?id={id}`	`Admin, Manager`
Schedules	DELETE	`/api/BusBooking/DeleteBusSchedule?id={id}`	`Admin`
Search	GET	`/api/BusBooking/searchBus?fromLocation={f}&toLocation={t}&travelDate={d}`	`AllowAnonymous`
Search	GET	`/api/BusBooking/searchBus2?fromLocation={f}&toLocation={t}&travelDate={d}`	`AllowAnonymous`
Bookings	GET	`/api/BusBooking/getBookedSeats?shceduleId={id}`	`AllowAnonymous`
Bookings	GET	`/api/BusBooking/GetAllBusBookings?vendorId={id}`	`Admin, Manager, Associate`
Bookings	GET	`/api/BusBooking/GetBusBooking?id={id}`	`Authenticated`
Bookings	POST	`/api/BusBooking/PostBusBooking`	`Authenticated`
Bookings	DELETE	`/api/BusBooking/DeleteBusBooking?id={id}`	`Authenticated`

Section 3: Step-by-Step Instructions to Run the Application Locally

3.1 Machine Prerequisites

- **.NET 10 SDK** (v10.0.100 or newer) — Verify by running `dotnet --version` in PowerShell.
- **Microsoft SQL Server** (Developer, Express, or Enterprise Edition running locally on `localhost`) and `sqlcmd` utility.
- **Windows PowerShell 5.1+** or **PowerShell 7+**.
- **IDE:** Visual Studio 2022/2026, VS Code with C# Dev Kit, or JetBrains Rider.

3.2 Step 1 — Open PowerShell & Navigate to the Workspace

```
cd C:\Projects\BusBookingApi
```

3.3 Step 2 — Verify Connection String & JWT Settings

Open `C:\Projects\BusBookingApi\src\BusBooking.Api\appsettings.json` and confirm the SQL Server instance name matches your local SQL Server installation:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=BusBookingDb;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "JwtSettings": {
    "Secret": "SuperSecretKeyForBusBookingApiMustBeAtLeast32BytesLong!",
    "Issuer": "BusBookingApi",
    "Audience": "BusBookingClient",
    "ExpiryMinutes": 120
  }
}
```

3.4 Step 3 — Initialize & Seed the SQL Server Database

Run the DDL and seed script using `sqlcmd` (creates `BusBookingDb`, all 7 tables, covering indexes, the `dbo.vw_ActiveScheduleSeats` indexed view, sample locations, and vendors):

```
sqlcmd -S localhost -E -i
"C:\Projects\BusBookingApi\scripts\01_Initial_Schema_And_Covering_Indexes.sql"
```

Alternatively, you can apply EF Core migrations directly:

```
dotnet ef database update --project
"src\BusBooking.Infrastructure\BusBooking.Infrastructure.csproj" --startup-project
"src\BusBooking.Api\BusBooking.Api.csproj"
```

3.5 Step 4 — Build & Run the Application

Build the full solution and start the API server on port `5241`:

```
# 1. Build the solution
dotnet build "C:\Projects\BusBookingApi\BusBooking.slnx"

# 2. Start the Web API
dotnet run --project "C:\Projects\BusBookingApi\src\BusBooking.Api\BusBooking.Api.csproj" --
urls "http://localhost:5241"
```

If setting up the database for the first time, seed the default `Admin`, `Manager`, and `Associate` accounts by running:

```
powershell -ExecutionPolicy Bypass -File
"C:\Projects\BusBookingApi\scripts\seed_users_via_api.ps1"
```

3.6 Step 5 — Using Swagger UI & Default Seeded Role Credentials

1. Open your browser to **`http://localhost:5241/swagger`** (or `http://localhost:5241/`).
2. Use any of the default seeded accounts below to test Role-Based Access Control:

Role	Username	Password	Email	Permitted Operations
Admin	`admin`	`Admin@123`	`admin@busbooking.local`	Full system access across all endpoints including all `DELETE` operations.
Manager	`manager`	`Manager@123`	`manager@busbooking.local`	View users; create, update, and view vendors, locations, and schedules.
Associate	`associate`	`Associate@123`	`associate@busbooking.local`	View vendors, schedules, and bookings; create/cancel customer bookings.

> **How to Authorize in Swagger UI:**
>
> 1. Expand `POST /api/Auth/login`, click **Try it out**, and execute with `{ "userName": "admin", "password": "Admin@123" }`.
> 2. Copy the JWT string from `data.token` in the response body.
> 3. Click the **Authorize** button at the top right of Swagger UI.
> 4. Paste your token in the **Value** box (works with either raw `<token>` or `Bearer <token>`) and click **Authorize**.
> 5. Execute protected endpoints such as `GET /api/Auth/GetAllUsers` or `GET /api/BusBooking/GetBusVendors`.

3.7 Step 6 — Running the Automated PowerShell Verification Suite

Stop any manually running `BusBooking.Api` process, then run the automated test scripts from PowerShell:

```
# Run full 8-step Swagger, Authentication & RBAC verification suite
powershell -ExecutionPolicy Bypass -File
"C:\Projects\BusBookingApi\scripts\test_auth_and_swagger.ps1"

# Run quick Admin login verification
powershell -ExecutionPolicy Bypass -File
"C:\Projects\BusBookingApi\scripts\verify_admin_login.ps1"

# Run health checks (/health/live, /health/ready) & locations verification
powershell -ExecutionPolicy Bypass -File
"C:\Projects\BusBookingApi\scripts\test_full_system.ps1"
```

3.8 Step 7 — Troubleshooting Guide for New Engineers

- **Build Error (`BusBooking.Api.exe is locked by another process`):** Stop the running API process before compiling:
  `Get-Process -Name "BusBooking.Api" -ErrorAction SilentlyContinue | Stop-Process -Force`
- **PowerShell 5.1 `Invoke-WebRequest` Hanging:** Always include `-UseBasicParsing` when using `Invoke-WebRequest` in Windows PowerShell 5.1 so it does not invoke the legacy Internet Explorer MSHTML DOM parser.
- **Swagger UI `401 Unauthorized` After Restart:** Press `Ctrl+F5` in your browser to ensure Swagger UI reloads the latest `/swagger/v1/swagger.json` security definition.